DOCUMENTO TÉCNICO OFICIAL & ARQUITECTURA

EngDrill Pro

Manual de Arquitectura, Funcionamiento Interno del Código y Guía Pedagógica (Niveles A2+ a B2+ Advanced)

Este documento explica en detalle cómo está construida la aplicación **EngDrill Pro** por dentro: cómo funciona su arquitectura *client-side first*, su generador combinatorio infinito sin coste de servidores, su motor de evaluación tolerante a la ortografía informal de trabajo, su algoritmo de Repetición Espaciada (SRS), y su integración serverless con modelos de Inteligencia Artificial (Groq / OpenAI / Llama 3).

⚡ Zero-Backend / 100% Client-Side

Toda la lógica de evaluación, combinatoria gramatical y persistencia de fallos corre en el navegador en JavaScript puro sin dependencias de base de datos externas.

🎯 Enfoque de Machaque A2+ a B2+

Sin distracciones de audios ni tests para niños: 100% ejercicios activos de gramática Cambridge, phrasal verbs, collocations y redacción en situaciones reales.

🧠 Motor de Tolerancia Ortográfica

Algoritmo con distancia Levenshtein que jamás penaliza guiones (`second hand` vs `second-hand`), apóstrofes o erratas menores de teclado.

📱 PWA Offline & Despliegue en 2 Clics

Service Worker v5 con soporte para instalación en iPhone y Android sin pasar por App Store y 0 € de coste de alojamiento en GitHub Pages.

Repositorio Oficial: github.com/adrianSG25/aprende-en
URL en Producción: https://adriansg25.github.io/aprende-en/
Tecnologías: React 19, TypeScript, Vite, Tailwind CSS v4, Web Workers

Versión: 5.0.0 (Release Non-Pedantic)
Fecha: Octubre 2026
Estado: Producción Activa

Tabla de Contenidos

1. Visión General y Filosofía de Diseño	Pág. 2
2. Arquitectura Global del Sistema (Por Dentro)	Pág. 3
3. El Motor de Evaluación y Tolerancia Ortográfica	Pág. 4
4. El Generador Infinito de Ejercicios (Sin IA / Cero Coste)	Pág. 5
5. Capa de Inteligencia Artificial Serverless (IA Opcional)	Pág. 6
6. Persistencia y Algoritmo de Repetición Espaciada (SRS)	Pág. 7
7. Recorrido por el Código Fuente (Archivo por Archivo)	Pág. 8
8. Cómo Explicar la App a Otra Persona (El Pitch de 2 Minutos)	Pág. 9
9. Despliegue, PWA y Acceso Móvil	Pág. 10

1. Visión General y Filosofía de Diseño

La mayoría de las aplicaciones de aprendizaje de idiomas del mercado (como Duolingo o Babbel) están orientadas a principiantes absolutos (A0–A1), utilizan mecánicas infantiles con animaciones lentas y dedican demasiado tiempo a audios básicos. Cuando un estudiante ya tiene un nivel intermedio (A2) y necesita consolidar las estructuras complejas del **B1 y B2** (inversiones sintácticas, condicionales mixtos, preposiciones dependientes y más de 60 phrasal verbs imprescindibles), este tipo de herramientas se vuelve ineficaz.

EngDrill Pro se diseñó bajo una premisa pedagógica radicalmente diferente:

- **Cero audios ni pérdida de tiempo pasiva:** El input auditivo se adquiere de forma natural viendo series o podcasts en versión original. La app se concentra al 100% en la *producción activa y la fijación de patrones gramaticales por repetición masiva*.
- **Entrenamiento por patrones y machaque:** Los hispanohablantes suelen cometer siempre los mismos 30 errores estructurales (ej. decir "*I am agree*" en vez de "*I agree*", o "*married with*" en vez de "*married to*"). EngDrill Pro cuenta con matrices específicas para desactivar esos errores mediante memoria muscular de tecleo.
- **Inmediatez absoluta:** Tiempo de respuesta menor a **5 milisegundos**. Cada ejercicio se evalúa al instante, mostrando en rojo la palabra fallada y en verde la solución correcta con su regla gramatical explicada en español.
- **Simulaciones reales de trabajo y vida cotidiana:** En la sección de redacción libre y situaciones ("¿Qué le responderías?"), el usuario se enfrenta a imprevistos reales (negociar un presupuesto con un mecánico en Londres, quejarse al casero por una fuga de agua, pedir explicaciones en un aeropuerto).

2. Arquitectura Global del Sistema (Por Dentro)

EngDrill Pro está desarrollada como una **Single Page Application (SPA) / Progressive Web App (PWA)** en **React 19 con TypeScript estricto**, compilada mediante el empaquetador ultrarrápido **Vite** y estilizada con **Tailwind CSS v4**.



¿Por qué esta arquitectura es superior para el usuario?

1. **Coste Cero de Servidor (0 €/mes):** Al no existir un backend en Node, Python o PHP que deba procesar cada clic, el proyecto se aloja como archivos estáticos puros en GitHub Pages. Puede atender a 1 persona o a 50.000 personas simultáneas sin que se caiga y sin pagar ni un solo céntimo.
2. **Privacidad Total:** Los datos del usuario (estadísticas, errores cometidos, racha y clave API de IA si la usa) se almacenan **únicamente en el propio dispositivo del usuario** a través de `localStorage`. No hay bases de datos externas recopilando datos personales.
3. **Funcionamiento Offline:** Gracias al `Service Worker v5`, una vez abierta la app en el móvil o portátil, el 90% de las funciones (todos los ejercicios de gramática, phrasal verbs, vocabulario y exámenes) funcionan en un avión o en el metro sin tener conexión a internet.

3. El Motor de Evaluación y Tolerancia Ortográfica

Uno de los problemas más frustrantes de las plataformas de idiomas convencionales es su **pedantería ortográfica**: si un usuario escribe `second hand` en lugar de `second-hand`, o si teclea rápido en el móvil y pone `dont` en vez de `don't`, muchas apps le dan la respuesta por errónea y le restan vidas.

En EngDrill Pro implementamos un motor de evaluación inteligente en dos capas dentro de `src/data/infiniteGenerator.ts` y `src/services/aiService.ts`:

1. Normalización Lingüística No Pedante (`normalizeAnswer`)

```
export function normalizeAnswer(raw: string): string {
  let s = raw.toLowerCase().trim();

  // 1. Los guiones se tratan exactamente igual que espacios (second-hand === second hand)
  s = s.replace(/[-–—_]/g, ' ');

  // 2. Expansión tolerante de contracciones sin apóstrofe para tecleo móvil
  s = s.replace(/\bdont\b/g, "don't").replace(/\bcant\b/g, "can't")
       .replace(/\bive\b/g, "i've").replace(/\bim\b/g, "i'm");

  // 3. Eliminación de signos de puntuación periféricos y espacios dobles
  s = s.replace(/[.,!?;:"'«»()[]]/g, '').replace(/\s+/g, ' ').trim();
  return s;
}
```

2. Tolerancia a Erratas Menores de Teclado (Fuzzy Matching con Levenshtein)

Cuando el usuario teclea palabras largas (5 letras o más), la función `checkAnswerMatch` calcula la distancia de edición de Levenshtein. Si el usuario escribió por ejemplo `responsible` pero se le escapó un dedo y puso `responsable`, o `explained` vs `expalined`, el evaluador local lo acepta pero le muestra en la interfaz la corrección visual exacta para que no se frustre por una simple pulsación de tecla.

> **✓ Filosofía de Corrección en Situaciones Reales**
>
> En un entorno laboral o cotidiano en Londres o Nueva York, escribir "*second hand part*" o "*dont worry*" en un correo o mensaje es 100% comprensible y natural. La app solo penaliza fallos **gramaticales y conceptuales reales** (como usar un tiempo verbal incorrecto o una preposición dependiente equivocada).

4. El Generador Infinito de Ejercicios (Sin IA / Cero Coste)

El corazón de la práctica masiva de la aplicación reside en el archivo `src/data/infiniteGenerator.ts`. En lugar de hacer que cada ejercicio dependa de una llamada a una IA externa (lo que sería lento, costoso y requeriría conexión continua), la app cuenta con un **motor combinatorio procedimental**.

Tipo de Ejercicio	Estructura en el Código	Objetivo Pedagógico Cambridge
Cloze / Relleno	`type: 'cloze'` Oración con hueco `___` y 4 opciones o respuesta abierta.	Preposiciones dependientes ("*accused of*", "*congratulate on*") y partículas de phrasal verbs.
Corrección de Errores	`type: 'error_correction'` Muestra una frase con una trampa típica de hispanohablantes.	Detección activa del fallo ("*Most of people*" ➔ "*Most people*"; "*explain me*" ➔ "*explain to me*").
Transformación de Oraciones	`type: 'transformation'` Oración base + Palabra Clave (KEY WORD) en mayúsculas.	Equivalente al ejercicio estrella de Cambridge B2 First (Use of English Part 4). Ejemplo: *AS* ➔ *not as fast as*.
Formación de Palabras	`type: 'cloze'` con raíz léxica en mayúsculas (ej. *SUCCEED* ➔ *success / successfully*).	Familias de sufijos y prefijos (Use of English Part 3): sustantivos, adjetivos, adverbios y opuestos con *un-*, *dis-*.

Algoritmo de Mezcla Barajada (Fisher-Yates)

La función `generateInfiniteDrillQueue(filterLevel, filterCategory)` toma el banco estático curado más las matrices combinatorias procedimentales, filtra por nivel y aplica un algoritmo de barajado *Fisher-Yates* garantizando que nunca aparezcan dos ejercicios idénticos seguidos, ofreciendo una experiencia infinita de práctica continua.

5. Capa de Inteligencia Artificial Serverless (IA Opcional)

Toda la lógica de IA se encuentra aislada en `src/services/aiService.ts`. Se conecta directamente desde el navegador del usuario a cualquier proveedor con formato OpenAI compatible (por defecto **Groq Cloud** por su extrema velocidad y generoso plan gratuito, pero también OpenAI, OpenRouter o servidores locales como Ollama).

Innovación Técnica 1: Auto-Descubrimiento de Modelos en Vivo

Un fallo común en apps que usan APIs es que cuando una empresa da de baja un modelo (por ejemplo, al migrar de *llama-3.1-70b* a *llama-3.3-70b*), la app arroja un error `404 model_not_found`. En EngDrill Pro implementamos la función:

```
export async function fetchAvailableModelsForKey(apiKey: string): Promise<string[]> {
  // Consulta el endpoint GET /v1/models de la API en vivo con la clave del usuario
  const res = await fetch('https://api.groq.com/openai/v1/models', {
    headers: { 'Authorization': `Bearer ${apiKey.trim()}` }
  });
  const data = await res.json();
  // Filtra automáticamente los modelos de chat activos y vigentes
  return data.data.map(m => m.id).filter(id => !id.includes('whisper'));
}
```

Innovación Técnica 2: Evaluación No Pedante por IA en Situaciones Reales

En la función `evaluateRoleplayResponseWithAI` (empleada en el simulador de "¿Qué le responderías?"), el prompt del sistema y el código TypeScript tienen reglas estrictas:

- El prompt ordena explícitamente al LLM: "*NEVER flag missing hyphens (e.g. 'second hand' vs 'second-hand'), missing apostrophes in contractions, or minor spelling slips as errors*".
- Posteriormente, el código ejecuta la función `isPedanticOrMinorOrthography` como filtro de seguridad en TypeScript sobre la lista de errores devuelta por la IA. Si la IA devuelve un error tiquismiquis, **el código lo descarta y eleva la nota a 9.5/10**.
- Además, genera automáticamente dos elementos de alto valor pedagógico:
  1. **Versión B2 Native Upgrade:** Cómo diría esa misma frase un nativo británico o americano usando colocaciones más ricas.
  2. **Character Follow-Up:** Qué responde el personaje a continuación (ej. qué replica el mecánico o casero) para permitir una conversación continua.

6. Persistencia y Algoritmo de Repetición Espaciada (SRS)

En `src/services/storageService.ts` reside la lógica del **Baúl de Errores (Mistake Vault)**. Aprender un idioma no consiste en acertar a la primera por casualidad, sino en transformar los errores sistemáticos en hábitos automáticos.



La Regla de las 3 Victorias Consecutivas

Cuando el usuario entra en la pestaña "*Baúl de Errores*", solo se le presentan los ejercicios en los que ha tropezado. Para que un ejercicio salga del baúl y se marque como `mastered: true`, el usuario está obligado a **acertarlo 3 veces consecutivas**. Si lo acierta 2 veces pero al tercer día vuelve a fallar, el contador `consecutiveCorrect` se resetea inmediatamente a `0`.

Control de Racha Diaria (Streak)

La app compara la fecha actual (`YYYY-MM-DD`) con `lastActiveDate`. Si el usuario practicó ayer, incrementa `streakDays + 1`. Si dejó pasar más de 48 horas sin practicar, la racha vuelve a 1 día, incentivando el hábito diario de tecleo.

7. Recorrido por el Código Fuente (Archivo por Archivo)

A continuación se describe la responsabilidad exacta de cada archivo clave dentro del directorio `src/`:

Archivo	Responsabilidad y Funciones Clave
`src/types/index.ts`	Contrato de datos de todo el proyecto en TypeScript. Define los niveles (`A2+` , `B1` , `B1+` , `B2` , `B2+`), las interfaces `Exercise` , `MistakeRecord` , `UserStats` , `RoleplayScenario` y `ExamResult` .
`src/App.tsx`	Componente raíz. Gestiona la barra de navegación superior (tabs), el estado global de la sesión, la racha diaria de fuego, el botón de ajustes y la orquestación de componentes.
`src/components/ExerciseCard.tsx`	Motor visual de renderizado de preguntas. Gestiona el campo de texto, el botón para mostrar la traducción al español, el resaltado de diferencias palabra por palabra (diff rojo/verde), y el botón de "Añadir al Baúl de Errores".
`src/components/DrillArena.tsx`	La arena de entrenamiento masivo. Permite alternar entre el modo infinito procedimental, el generador con IA por temas libres, y el Baúl de Errores para machaque intensivo.
`src/components/WritingZone.tsx`	Módulo de redacción y situaciones reales ("¿Qué le responderías?"). Contiene escenarios preconfigurados (el mecánico, el casero, el trabajo) y generador de nuevos escenarios interactivos.
`src/components/ReadingZone.tsx`	Lector comprensivo con análisis sintáctico "Rayos X". Desglosa los párrafos de historias B1/B2 señalando las construcciones gramaticales utilizadas y lanza preguntas abiertas de redacción.
`src/components/PhrasalVerbsZone.tsx`	Explorador de 60+ Phrasal Verbs organizados por la *lógica de la partícula* (ej. *OFF* = desconexión/salida; *UP* = finalización completa) con 3 modos de práctica.
`src/components/ExamZone.tsx`	Simulador oficial cronometrado Cambridge Use of English B1/B2 y test diagnóstico de nivelación con desglose de porcentajes por categoría gramatical.
`src/public/sw.js`	Service Worker v5. Intercepta las peticiones de red y cachea todos los ficheros JS, CSS, iconos y HTML para arranque instantáneo y uso sin internet.

8. Cómo Explicar la App a Otra Persona (El Pitch de 2 Minutos)

Si un amigo, compañero de trabajo, profesor o entrevistador te pregunta: "*¿Cómo funciona esta app de inglés por detrás?*", aquí tienes la explicación perfecta estructurada en 4 puntos clave:

> 🗣️ **Guion de Respuesta Rápida:**
>
> *"Es una Progressive Web App desarrollada en **React 19 y TypeScript** pensada para saltar del nivel A2 al B2 avanzado de Cambridge. A diferencia de las apps típicas, está 100% enfocada en machacar gramática y redacción de situaciones reales sin perder el tiempo en audios básicos.*
>
> *Por dentro funciona con una arquitectura **client-side first**: tiene un generador combinatorio con más de mil ejercicios algorítmicos que se ejecutan en local a cero coste y en menos de 5 milisegundos, sin necesidad de servidores. Tiene un sistema de **repetición espaciada** que te obliga a acertar cada fallo 3 veces consecutivas para darlo por superado.*
>
> *Además, cuenta con una integración serverless con **modelos de lenguaje (IA)** que evalúa respuestas libres y te pone en situaciones cotidianas (como defenderte en el taller mecánico en inglés). El evaluador está programado con un algoritmo de **tolerancia ortográfica**: no es pedante con guiones ni apóstrofes y se centra en enseñarte inglés real y práctico para trabajar."*

Preguntas Frecuentes que te pueden hacer y cómo responder:

- **"¿Cuánto cuesta mantener la app online?"**
  0 euros al mes. Está alojada de forma estática en GitHub Pages y compilada con Vite. No requiere ningún servidor en la nube de pago.
- **"¿Cómo hace para no dar error si pongo 'second hand' sin guion?"**
  El código pasa cualquier respuesta por una función llamada `normalizeAnswer` *que unifica guiones con espacios y tolera contracciones informales, y si se usa la IA, un filtro post-procesador descarta cualquier corrección tiquismiquis antes de mostrarla.*
- **"¿Cómo se instala en el móvil si no está en la App Store?"**
  Es una PWA (Progressive Web App). Al abrir el enlace desde Safari en iPhone o Chrome en Android, pulsas 'Compartir' o menú de 3 puntos y seleccionas 'Añadir a pantalla de inicio'. Se instala al instante como si fuera nativa y funciona a pantalla completa.

9. Despliegue, PWA y Acceso Móvil

Proceso de Compilación y Publicación en GitHub Pages

1. **Compilación con Typecheck:** Se ejecuta `npm run build`, que primero valida todos los tipos con `tsc -b` y luego genera el paquete optimizado y minificado con Vite en la carpeta `dist/`.
2. **Soporte SPA en Servidores Estáticos:** Se copia `dist/index.html` a `dist/404.html` para que las recargas directas o rutas profundas en GitHub Pages no provoquen error 404.
3. **Despliegue a la rama `gh-pages`:** El contenido de `dist/` se empuja mediante Git a la rama huérfana `gh-pages` del repositorio de GitHub, quedando inmediatamente disponible a nivel mundial bajo CDN de alta velocidad.

PWA Manifest y Service Worker v5

El archivo `public/manifest.json` declara el modo `display: "standalone"`, con tema oscuro (`#0f172a`) y los iconos vectoriales SVG. El `public/sw.js` implementa una estrategia de caché **Network-First con fallback a Cache Storage**:

```
// public/sw.js - Versión 5 de Caché PWA
const CACHE_NAME = 'engdrill-pro-v5';

self.addEventListener('activate', (event) => {
  // Limpia automáticamente versiones antiguas de caché cuando se sube una nueva release
  event.waitUntil(
    caches.keys().then((keys) =>
      Promise.all(keys.map(k => k !== CACHE_NAME ? caches.delete(k) : null))
    ).then(() => self.clients.claim())
  );
});
```

> 📌 **Conclusión y Resumen del Proyecto**
>
> EngDrill Pro combina lo mejor de la ingeniería de software moderna (TypeScript estricto, React 19, PWA offline, zero-backend serverless) con una metodología pedagógica basada en la práctica activa y el machaque de patrones de alto rendimiento para hispanohablantes.